Westin Gjervold

ComSci 342 Homework 7

1. (8 pt) [Type Checking]: Using the new type syntax from TypeLang, provide the type of the expression. If the expression fails to type check, provide a brief description as to why type checking would fail.
    a. (lambda (x : num) (lambda (y : num) #t))

Inner lambda takes a num and returns #t, which is a bool. The outer lambda takes a num and returns the inner function, so the type is bool.

    b. (lambda (x : num) (lambda (y : num) (if x #t #f)))

The if condition is x, but x is a num and the if condition must be a bool, therefore there is a Type error.

    c. (lambda (x : num) (lambda (y : num) (if (> x y) #t y)))

The two branches of the if statement must have the same type. The then branch is a bool, but the else is a num. Therefore, this is a type error due to the mismatched branches on the if statement.

    d. (let ((x : num 8) (y : bool #f)) (cons (cons x y) (list : (num , bool))))

x : num and y : bool so (cons x y) produces pair (num, bool)

(list : (num, bool)) is empty list of type List<(num, bool)>

Cons (cons x y) (list : (num, bool))) prepends a (num, bool) pair onto a List<(num, bool)>

Type is List<(num, bool)>

2. (5 pt) [TypeLang Programming for Lists]: In HW4, you have written a function compList that takes a list and remove consecutive repetitive numbers. Rewrite the function in TypeLang.

```
(define compList : (List<num> -> List<num>)
                 (lambda (lst : List<num>)
                   (if (null? lst)
                     (list : num)
                     (if (null? (cdr lst))
                       lst
                       (if (= (car lst) (car (cdr lst)))
                         (compList (cdr lst))
                         (cons (car lst) (compList (cdr lst))))
                       )
                     )
                   )
  )
```

3. (5 pt) [TypeLang Programming for Pairs]: A function to swap variable values can be defined by a lambda function that takes a single pair (a b) and returns a new pair (b a). Instead of taking in a pair of values, we can take in a pair holding references to where a and b are stored and the function updates the referenced values. Write a function swap in which you accept a pair of references and swaps the values referenced and returns the pair of references.

```
(define swap : ((Ref num ,Ref num) -> (Ref num ,Ref num))
               (lambda (p : (Ref num ,Ref num))
                 (let ((a : num (deref (car p)))
                       (b : num (deref (cdr p))))
                   (let ((dummy1 : num (set! (car p) b)))
                     (let ((dummy2 : num (set! (cdr p) a)))
                       p
                       )
                     )
                   )
                 )
  )
```

4. (8 pt) [Memory Management]: Modify the TypeLang to change the internal implementation of the heap used to store the dynamic memory and handle dynamic memory allocation.

I removed the index-based allocation and replaced it with a loop that scans the heap for the first available (null) slot. This change allows freed memory locations to be reused instead of always allocating new ones sequentially.

```
public Value ref(Value value) {
    for (int i = 0; i < HEAP_SIZE; i++) {
        if (_rep[i] == null) {
            Value.RefVal new_loc = new Value.RefVal(i);
            _rep[i] = value;
            return new_loc;
        }
    }
    return new Value.DynamicError("Out of memory error");
}
```

5. (16 pt) [Memory Management, Type System]: The second modification you will make to TypeLang will add a new operator which checks if two expressions refer to the same memory address.

TypeLang.g4:

Added exp rules in TypeLang file with the new refeq rule:

```
exp returns [Exp ast]:
    va=varexp         { $ast = $va.ast; }
  | num=numexp        { $ast = $num.ast; }
  | bl=boolexp        { $ast = $bl.ast; }
  | add=addexp        { $ast = $add.ast; }
  | sub=subexp        { $ast = $sub.ast; }
  | mul=multexp       { $ast = $mul.ast; }
  | div=divexp        { $ast = $div.ast; }
  | let=letexp        { $ast = $let.ast; }
  | lam=lambdaexp     { $ast = $lam.ast; }
  | call=callexp      { $ast = $call.ast; }
  | i=ifexp           { $ast = $i.ast; }
  | less=lessexp      { $ast = $less.ast; }
  | eq=equalexp       { $ast = $eq.ast; }
  | gt=greaterexp     { $ast = $gt.ast; }
  | car=carexp        { $ast = $car.ast; }
  | cdr=cdrexp        { $ast = $cdr.ast; }
  | cons=consexp      { $ast = $cons.ast; }
  | list=listexp      { $ast = $list.ast; }
  | nl=nullexp        { $ast = $nl.ast; }
  | ref=refexp        { $ast = $ref.ast; }
  | deref=derefexp    { $ast = $deref.ast; }
  | assign=assignexp { $ast = $assign.ast; }
  | free=freeexp      { $ast = $free.ast; }
  | refeq=refeqexp    { $ast = $refeq.ast; }
  ;
```

Added refeqexp returns:

```
refeqexp returns [RefEqExp ast] :
    '(' '==' e1=exp e2=exp ')' { $ast = new RefEqExp($e1.ast, $e2.ast); }
    ;
```

AST.java:

Added new RefEqExp class:

```
class RefEqExp extends Exp {
    2 usages
    private final Exp _first_exp;
    2 usages
    private final Exp _second_exp;
    2 usages
    public RefEqExp(Exp first_exp, Exp second_exp) {
        _first_exp = first_exp;
        _second_exp = second_exp;
    }
    no usages
    public Exp first_exp() {
        return _first_exp;
    }
    no usages
    public Exp second_exp() {
        return _second_exp;
    }
    93 usages
    public <V, T> V accept(Visitor<V, T> visitor, Env<T> env) {
        return visitor.visit( e: this, env);
    }
}
```

Added RefEqExp to the visitor interface:

```
1 usage
V visit(AST.RefEqExp e, Env<T> env);
```

Evaluator.java:

Added the new RefEqExp visit method:

```
public Value visit(RefEqExp e, Env<Value> env) {
    Value v1 = e.first_exp().accept( visitor: this, env);
    Value v2 = e.second_exp().accept( visitor: this, env);
    if (v1 instanceof Value.RefVal && v2 instanceof Value.RefVal) {
        return new Value.BoolVal(v1.equals(v2));
    }
    return new Value.DynamicError("RefEqExp requires two reference values");
}
```

Printer.java:

Added visit method:

```
@Override
public String visit(AST.RefEqExp e, Env<T> env) {
    return "(== %s %s)".formatted(
            e.first_exp().accept( visitor: this, env),
            e.second_exp().accept( visitor: this, env)
    );
}
```

Checker.java:

Added new type checking visit method:

```
public Type visit(RefEqExp e, Env<Type> env) {
    Type t1 = e.first_exp().accept( visitor: this, env);
    Type t2 = e.second_exp().accept( visitor: this, env);
    if (!(t1 instanceof RefT)) {
        return new ErrorT("RefEqExp requires a reference, found " + t1);
    }
    if (!(t2 instanceof RefT)) {
        return new ErrorT("RefEqExp requires a reference, found " + t2);
    }
    return BoolT.getInstance();
}
```